TP sur les signaux

Exercise 1 :

a. Write a program in c which displays the famous message "Hello, World!"
b. Now modify the program so that the father creates a child that displays the message “Hello,” and the father displays the message “world!”. Test your program. What do you notice?
c. Modify the program using the wait () primitive to ensure that the message is displayed in the correct order.
d. Let’s complicate the problem. Modify the problem by reversing the roles of father and son i.e. the son displays the message "world!" and the father displays the message "Hello,".Use the primitives: kill (), signal () and pause () to ensure that the message is displayed in the correct order...

Exercise 2 :

a. Write a program that does a "fork ()". The son sends a "SIGUSR1" signal to the father 20 times. Each time the father receives the "SIGUSR1" signal, he displays on the screen: "I am the father, I have received a SIGUSR1 signal from my son for the Nth time" (with N the number of times the signal is received). After the 20 signal sends, the wire ends. The father then displays "My child process with pid number X has just terminated, and returned an error code Y", and the father terminates. Start this program. What are you observing? Why ?
b. Modify the code of the son so as not to send a "SIGUSR1" signal 20 times, but 200,000 times. What's going on ? Why ?
c. Modify the child again to only send the "SIGUSR1" signal to the father 20 times, but this time, in the child code, after the call to the "kill ()" function, do a "sleep (1) " What's going on ? Why ?

Exercice 3 :

a. Write a program that stops after receiving the SIGINT signal 5 times.
b. Write a program that ignores the SIGINT signal.
c. Write a program that displays numbers between 0 and 20 using two processes (for example, the parent process will display even numbers, the child will display odd numbers)..

Exercice 4 :

a. Write an invincible program that says “ouch!” when it receives SIGINT and continues to run.
b. Make it give up after p attempts (value given in parameter):

```
gim@ytterbium $ ./invincible 5
(^C) aie!
(^C) aie!
(^C) aie!
(^C) aie!
(^C) ca suffit, j’ai trop mal...
gim@ytterbium $
```

Exercise 5 :

Write a program that continuously displays the "hello" message, while displaying the "goodbye" message every three seconds.

Exercice 6 :

The void alarm (int sec) function is a function that instructs the operating system to send a SIGALRM signal to the process after sec seconds. To deactivate the alarm, we use alarm (0); Write a program that asks the user to type in their first name on the keyboard, and quits after ten (thirty) seconds if the user has not typed anything, displaying "too late!".

Exercise 7 :

Write a program creating two sons, sending the SIGUSR1 signal to his younger son.
Upon receipt of this signal, the younger son should send the SIGUSR2 signal to the eldest son (which causes his termination) before stopping.

Exercise 8 :

Write a program that intercepts CTRL-C and only accepts to kill the process corresponding to that program after verification with a password.

Exercise 9 :

Write a small program which intercepts two signals: the user's C control (which tries to stop it) and the SIGUSR1 (coming from another terminal) which on the contrary prolongs its life. For example, it is necessary to have an integer (declared as a whole, therefore out of any function) of 5 at the start of the program. Each call to CTR-C decrements this counter, and reception of a SIGUSR1 signal adds 10. The hand for its part loops as long as the number of lives is not equal to zero.

Exercise 10 :

Write a program that:

1. processes all signals with a handler function which displays the number of the received signal.
2. process the SIGUSR1 signal with a handler1 function and the SIGUSR2 signal with handler2:
    a. handler1 displays the number of the signal received and the list of users of the machine (call to who by system ("who"))
    b. handler2 displays the number of the received signal and the disk space used on the machine (call to df. by system ("df."))
3. Start the program and send it signals, including SIGUSR1 and SIGUSR2, from a shell, using the kill command.